CS683: Advanced Computer Architecture

Programming Assignment 1

Team Members

Abhinav V	24B1007
Boominadharshan K	24B1225
Sabarinath S	24B1052
Santhosh BV	24B1290

Contents

1 Task-1 **1**
1.1 Loop Unrolling 1
1.2 Loop Reordering 2
1.3 Tiling 2
1.4 SIMD 3
1.5 Multiple Optimization 5

1 Task-1

1.1 Loop Unrolling

Looking at the naive convolution implementation, we can see that each iteration of the innermost loop involves a comparison, branch instruction, and address calculation operations, adding significant loop overhead. Loop unrolling essentially reduces this overhead by executing multiple operations within a single iteration, reducing the number of loop-control instructions and exposing more instruction-level parallelism. Thus, each broadcasted kernel coefficient is multiplied with multiple input matrix terms, while the intermediate partial sums are stored in multiple accumulators and the results are later written to the output matrix.

For different lengths of loop unrolling, speedup, branch operations, branch miss rate was compared against the naive convolution implementation,(n is the number of unrolling factor(along row as well as column)),

Table 1: Performance metrics for different loop unrolling factors

Metric	**Naive**	**Unroll ×2**	**Unroll ×4**	**Unroll ×6**
Execution time (ms)	16.292	11.952	8.304	7.552
Instructions	331,382,975	216,036,562	129,265,370	101,990,323
Cycles	55,536,828	39,138,531	28,456,732	24,697,938
Branches	58,724,401	29,362,219	14,681,137	9,777,879
Branch misses	2,483	2,508	3,521	2,920
IPC	5.967	5.520	4.543	4.130
Branch miss rate (%)	0.0042	0.0085	0.0240	0.0299
Speedup	1.000×	1.363×	1.962×	2.185×
Instruction reduction (%)	0.00	34.81	60.99	69.22
Branch reduction (%)	0.00	50.00	75.00	83.35

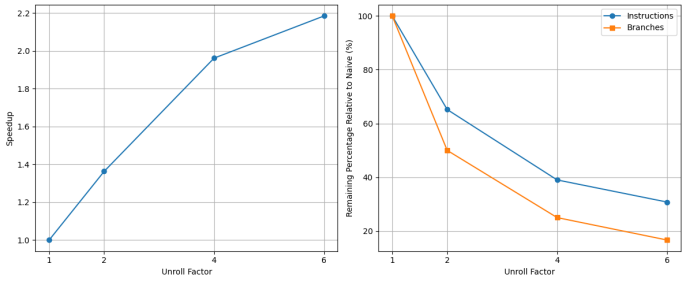


Figure 1: Time vs Unrolling factor

A saturation effect is observed with an increase in the loop unrolling factor. This is because increasing n only reduces loop-control overhead and the number of branch instructions. A significant

number of instructions remain unaltered, corresponding primarily to the convolution computations, memory accesses, and other operations that must be performed regardless of the unrolling factor.

1.2 Loop Reordering

Changing the order of loops can drastically reduce cache miss rates by improving spatial locality in the innermost loop traversal. Various loop reordering configurations were experimented with, and the most effective ordering was found to be oy, ky, kx, ox(here oy,ox are the indices of the output matrix and ky,kx are the indices of the kernel matrix). With ox as the innermost loop, consecutive iterations access consecutive elements of the input and output matrices, as seen from the input index $(oy + ky) \times \text{stride} + (ox + kx)$. This improves spatial locality by allowing consecutive memory accesses to utilize the same cache lines more effectively.

Misses per thousand instructions, Instructions per cycles, speedup and execution time of the re-ordered code was observed against baseline code. In the reordered code, the access and indexing overhead associated with each kernel element is amortized over W as a single kernel element is loaded once and reused across an entire output row. However, the number of output matrix accesses increases, since the partial sums must be read, updated, and written back during each iteration of the inner loop. Additionally, the output matrix must be initialized to zero before the convolution operation. Consequently, the increased output memory accesses results in a higher MPKI. Nevertheless, the overall instruction count is reduced due to reduced loop overhead and kernel reuse, resulting in improved execution performance and an overall speedup.

Table 2: Performance comparison between naive and loop-reordered convolution

Metric	**Naive**	**Reordered**
Execution time (ms)	24.222	14.671
Instructions	331,382,961	264,587,431
Cycles	81,054,419	48,906,624
L1D misses	872,949	1,382,617
IPC	4.088	5.410
L1D MPKI	2.634	5.226
Speedup	1.000×	1.651×

1.3 Tiling

The L1D cache size is commonly observed to be 32 kB.

The tiling code divides the matrix into smaller tiles and implements naive convolution. The idea is that when one row of the output tile is computed and we move to the next row, the required input matrix elements are not kicked out of the cache yet and can thus be reused, reducing MPKI. Different tile sizes were experimented with to find the optimal tile size. Speedup from tiling is function of both the dimensions of tile, thus square tiles were experimented with for simpler analysis.

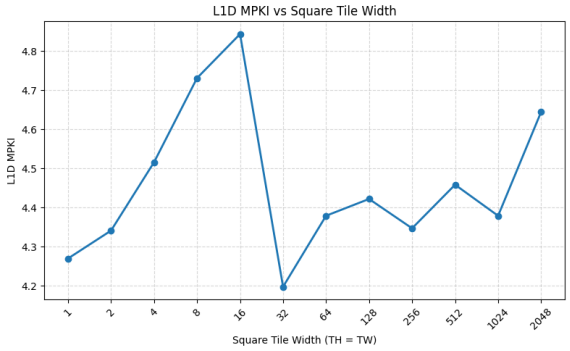


Figure 2: MPKI for different square tile sizes

Speedup achieved through tiling depends on several factors, including cache line size, cache associativity, and memory access patterns. Smaller tiles can improve cache locality and data reuse, but they also increase the number of tiles and associated loop overhead. An effective trade-off between locality, reuse, and overhead is typically observed for medium-sized tiles. As tile size increases further, the working set may exceed the cache capacity, resulting in cache evictions and reducing effective cache reuse.

1.4 SIMD

In our SIMD implementation, the output elements in each row are first grouped according to the SIMD register width, i.e., groups of 4, 8, or 16 single-precision floating-point elements depending on the register size used.

The convolution operation then follows the same traversal order as the naive implementation. However, instead of computing one output element at a time, a group of output elements is processed simultaneously using SIMD registers. For each kernel element, the corresponding contiguous input elements are loaded into a SIMD register, multiplied by the broadcasted kernel value, and accumulated in parallel.

Finally, if the row width is not completely divisible by the SIMD group size, the remaining elements (tail) are processed separately using the scalar naive convolution approach.

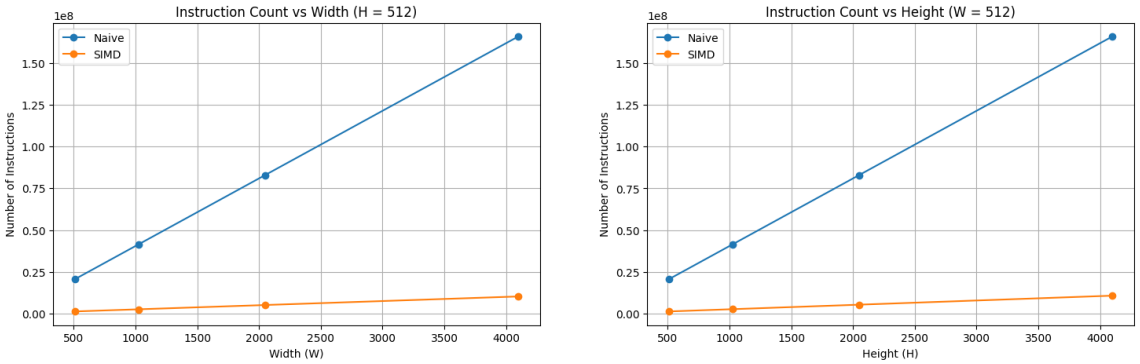
We compared various performance metrics for the baseline and SIMD-optimized implementations. As observed, the instruction count decreases drastically since multiple data elements are processed using a single SIMD instruction, reducing the number of load, addition, and multiplication instructions required. The wider data path for load operations, along with parallel execution of arithmetic operations, results in a significant improvement in execution speed.For 512SIMD registers,

Table 3: Performance comparison for a 2048×2048 input matrix.

Metric	**Naive**	**SIMD**
Instruction Count	331382880	20721755
Instructions/Output	79.01	4.94
Execution Time (ms)	20.201514	3.095874
Speedup	6.525×	

The height and width of the input matrix were varied independently while keeping the other dimension fixed. As expected, the total number of instructions increased with the size of the input matrix.

Since the AVX-512 implementation processes 16 elements at a time, any remaining elements when W is not a multiple of 16 are processed using the naive scalar implementation. Consequently,



(a) Instruction count as matrix width changed

(b) Instruction count as matrix length changed

Figure 3: Overall caption describing both images.

the instruction count per output increases for widths where $W \bmod 16 \neq 0$, due to the additional overhead of scalar tail processing.

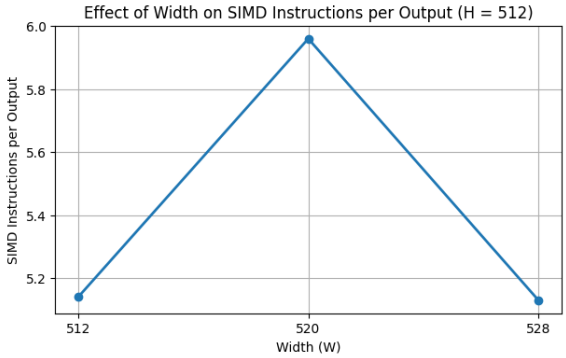


Figure 4: Instruction per output for matrix dimension with and without tail

Comparing the instruction count for same matrix size across different widths of simd registers,

Table 4: Comparison of metrics different SIMD register widths

SIMD Register Width	**Instruction Count**	**Instructions/Output**
128-bit	82,051,166	19.56
256-bit	41,164,895	9.81
512-bit	20,721,755	4.94

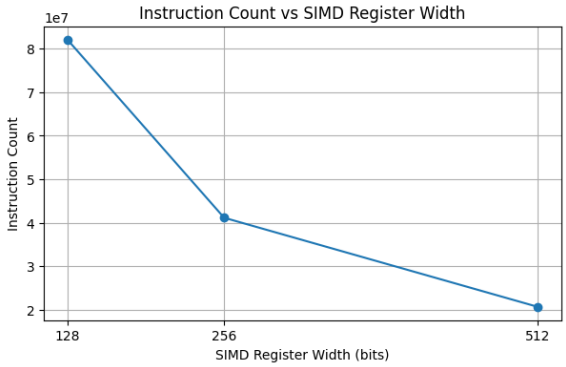


Figure 5: Instructions count for different widths

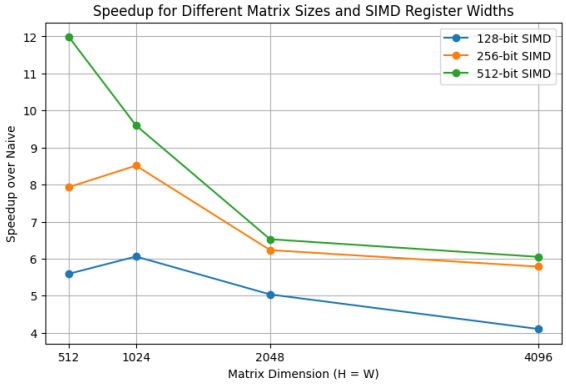


Figure 6: Speedup for different dimensions and widths

When the matrix size is 512, it requires 32 lines to store an entire row. A traversal through the width of the matrix for a particular height requires three rows. Since the L1D cache size is 32Kb this data can be fitted perfectly into the cache. When the convolution for the next row is calculated the earlier extracted last two rows are still in the cache and can be reused. When the size of the matrix increases, lines required to store the rows completely increases, thus there is a higher probability that the lines brought in earlier are evicted. Thus the speedup decreases as the size increases after a particular value, which is 1024 in this case.

1.5 Multiple Optimization

All the above techniques were overlayed to achieve synergetic performance gain. SIMD, loop unrolling and reordering techniques were used.

```
void conv_optimized(const float* in, float* out, const float* ker,
                    int H, int W, int K) {
    const int p = K / 2;
    const int in_stride = W + 2 * p;

    for (int oy = 0; oy < 4 * (H / 4); oy += 4) {
        for (int ox = 0; ox < W; ox += 8) {
            __m256 acc0 = _mm256_setzero_ps();
            __m256 acc1 = _mm256_setzero_ps();
            __m256 acc2 = _mm256_setzero_ps();
```

```
            __m256 acc3 = _mm256_setzero_ps();

            for (int kx = 0; kx < K; ++kx) {
                __m256 row0 = _mm256_loadu_ps(
                    &in[(oy + 0) * in_stride + (ox + kx)]);
                __m256 row1 = _mm256_loadu_ps(
                    &in[(oy + 1) * in_stride + (ox + kx)]);
                __m256 row2 = _mm256_loadu_ps(
                    &in[(oy + 2) * in_stride + (ox + kx)]);
                __m256 row3 = _mm256_loadu_ps(
                    &in[(oy + 3) * in_stride + (ox + kx)]);

                for (int ky = 0; ky < K; ++ky) {
                    __m256 ker_val =
                        _mm256_set1_ps(ker[ky * K + kx]);

                    acc0 = _mm256_fmadd_ps(row0, ker_val, acc0);
                    acc1 = _mm256_fmadd_ps(row1, ker_val, acc1);
                    acc2 = _mm256_fmadd_ps(row2, ker_val, acc2);
                    acc3 = _mm256_fmadd_ps(row3, ker_val, acc3);

                    row0 = row1;
                    row1 = row2;
                    row2 = row3;

                    if (ky < K - 1) {
                        row3 = _mm256_loadu_ps(
                            &in[(oy + 4 + ky) * in_stride
                                + (ox + kx)]);
                    }
                }
            }

            _mm256_storeu_ps(&out[(oy + 0) * W + ox], acc0);
            _mm256_storeu_ps(&out[(oy + 1) * W + ox], acc1);
            _mm256_storeu_ps(&out[(oy + 2) * W + ox], acc2);
            _mm256_storeu_ps(&out[(oy + 3) * W + ox], acc3);
        }
    }

    for (int oy = 4 * (H / 4); oy < H; ++oy) {
        for (int ox = 0; ox < W; ox += 8) {
            __m256 acc = _mm256_setzero_ps();

            for (int ky = 0; ky < K; ++ky) {
                for (int kx = 0; kx < K; ++kx) {
                    __m256 in1 = _mm256_loadu_ps(
                        &in[(oy + ky) * in_stride + (ox + kx)]);
                    __m256 ker1 =
                        _mm256_set1_ps(ker[ky * K + kx]);

                    acc = _mm256_fmadd_ps(in1, ker1, acc);
                }
            }

            _mm256_storeu_ps(&out[oy * W + ox], acc);
        }
```

```
    }
}
```

Listing 1: Optimized convolution using AVX vectorization and loop unrolling

The optimization combines SIMD vectorization with loop unrolling to process a 4×8 block of output elements at a time. SIMD registers are used across the column dimension, allowing eight contiguous floating-point elements to be processed simultaneously. The computation is further unrolled across four output rows using four independent accumulators, thereby extending the original SIMD implementation to three additional neighbouring rows.

The output matrix is traversed in 4×8 tiles from top-left to bottom-right. Within each tile, overlapping input data accessed by neighbouring output rows is reused during partial sum computation. As the convolution window progresses across the kernel rows, previously loaded 8-element input vectors are shifted and reused, requiring only one new vector to be loaded for subsequent iterations. This reduces redundant memory accesses while exposing additional instruction-level parallelism through the independent accumulators.

Overall, the implementation combines 8-wide SIMD parallelism with 4-way loop unrolling, computing the partial sums for 32 output elements simultaneously. This effectively provides register-level blocking, reducing the need for tiling.

Table 5: Performance comparison of optimization stages for different kernel sizes

Kernel	**Stage**	**Time (ms)**	**GFLOP/s**	**Speedup**	**Correct**
$K = 3$	Naive (Ref)	26.396	2.86	1.00×	Yes
	Reorder	15.539	4.86	1.70×	Yes
	Unroll	7.711	9.79	3.42×	Yes
	Tile	16.584	4.55	1.59×	Yes
	SIMD	2.990	25.25	8.83×	Yes
	Optimized	2.755	27.40	9.58×	Yes
$K = 5$	Naive (Ref)	72.978	2.87	1.00×	Yes
	Reorder	37.217	5.63	1.96×	Yes
	Unroll	19.579	10.71	3.73×	Yes
	Tile	45.389	4.62	1.61×	Yes
	SIMD	5.460	38.41	13.37×	Yes
	Optimized	4.615	45.45	15.81×	Yes

Apart from the individual optimization techniques, different combinations of the discussed techniques were also experimented with. A summary of these experiments is listed below:

- SIMD and Unrolling
- SIMD and Loop Reordering
- SIMD,Optimized Loading,Unrolling and Tiling
- Final Optimized Implementation

All these implementations were compared against the naive convolution implementation using various performance metrics, primarily execution time and speedup.

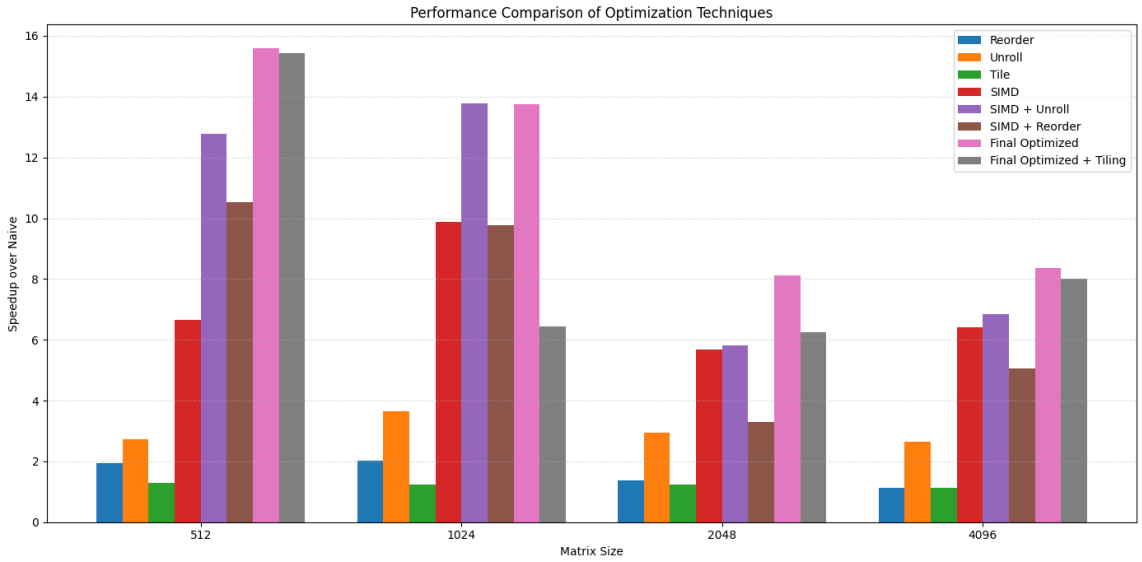


Figure 7: Performance of various techniques on different dimensions of input matrix

Our initial inference that tiling is rebundant on our final optimized code is evident from the above graph. We also analysed performance of different techniques for various kernel sizes,

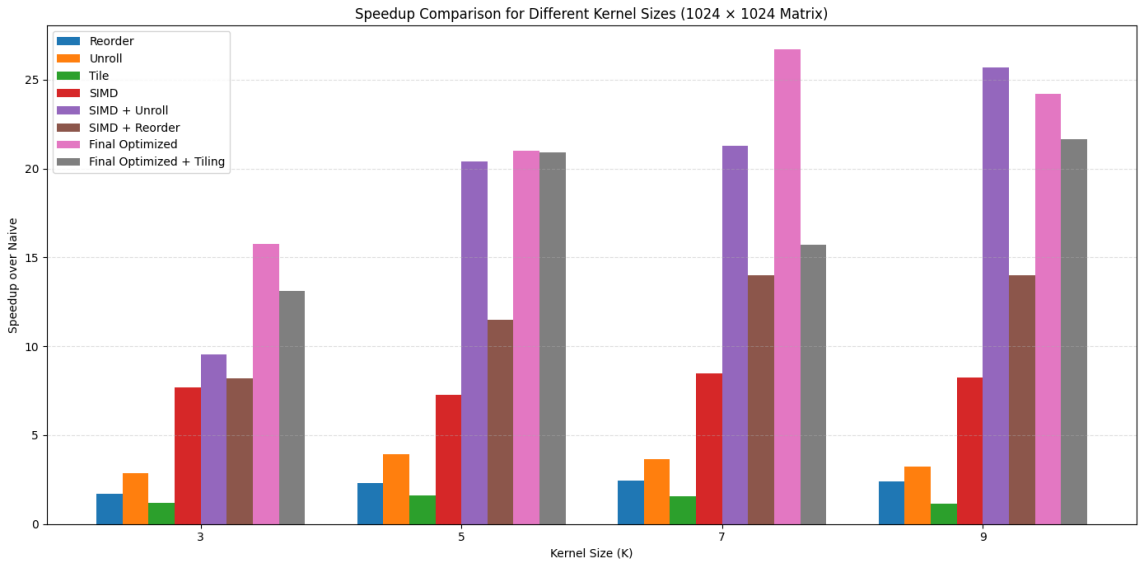


Figure 8: Performance of various techniques on different dimensions of kernel